

Đúng. Với **SE1944**, quy trình chuẩn là:

Tìm paper để có cơ sở → chọn/tạo dataset code → cho AI sinh test → cho sinh viên viết test thủ công → chạy coverage/mutation → so sánh → viết báo cáo.

Đề tài SE1944 của bạn là **LLM for Unit Test Case Generation**, cụ thể hỏi GPT-4 sinh unit test cho Java/Python functions có đạt **branch coverage ≥80%** và **mutation score ≥60%** so với test viết thủ công bởi sinh viên không.

File hướng dẫn cũng ghi flow tổng quát là: topic/RQ → PRISMA → evidence table → GAP → tình chỉnh RQ → thiết kế thực nghiệm → chạy thực nghiệm → viết paper.

Quy trình chi tiết cho nhóm 5 người SE1944

Giai đoạn 0 — Hiểu đề tài và chốt phạm vi

Mục tiêu của giai đoạn này là cả nhóm phải hiểu giống nhau: mình đang nghiên cứu **AI sinh unit test**, không phải AI sinh code app.

RQ của nhóm bạn nên hiểu thành:

Với các hàm Java/Python có độ phức tạp trung bình, GPT-4 tự sinh unit test có đạt branch coverage ≥80% và mutation score ≥60% không, khi so với unit test viết thủ công bởi sinh viên?

Chia PICO cho SE1944:

PICO	Nội dung của nhóm bạn
P — Population	Java/Python functions có cyclomatic complexity trung bình
I — Intervention	GPT-4 sinh unit test cases
C — Comparison	Unit test cases do sinh viên viết thủ công
O — Outcome	Branch coverage ≥80%, mutation score ≥60%

Output cần có ở giai đoạn này:

docs/research_question.md
docs/pico.md
docs/scope.md

Trong đó **scope.md** phải ghi rõ nhóm chọn **Java hay Python hay cả hai**. Với nhóm mới, mình khuyên chọn **Java trước**, vì có **JUnit + JaCoCo + PIT** rất hợp với branch coverage và mutation

score.

Giai đoạn 1 — Tìm paper bằng PRISMA

Đây là phần “đi tìm nguồn”. Nhưng nhớ: **tìm paper không phải là kết quả cuối cùng**, mà là để biết người ta đã làm gì và tìm GAP.

File hướng dẫn nói SLR là tìm paper có hệ thống, PRISMA là quy trình search → screen → include, evidence table là bảng tóm tắt paper, còn GAP là thứ chưa ai nghiên cứu rõ.

Bước 1.1 — Chọn database cho 5 người

File yêu cầu mỗi thành viên chọn một nguồn search, sau đó đối chiếu paper trùng, tạo bảng tổng hợp và GAP evidence final.

Nhóm 5 người chia như này:

Thành viên	Nguồn search	Việc cần làm
Người 1	Google Scholar	Tìm rộng nhất, lấy nhiều paper ban đầu
Người 2	IEEE Xplore	Tìm paper chính thống về testing/SE
Người 3	ACM Digital Library	Tìm paper ở ASE, FSE, ISSTA, ICSE
Người 4	Semantic Scholar	Tìm paper + PDF/preprint
Người 5	arXiv + CORE/ResearchGate	Tìm paper mới về LLM và bản PDF miễn phí

Bước 1.2 — Dùng search string cho SE1944

Không dùng search string Gherkin/BDD nữa. Nhóm bạn dùng các chuỗi như sau:

```
("large language model" OR "LLM" OR "GPT" OR "ChatGPT")  
AND ("unit test generation" OR "test case generation" OR "automated unit testing")  
AND ("branch coverage" OR "code coverage" OR "mutation score" OR "mutation testing")
```

```
("GPT" OR "ChatGPT" OR "large language model")  
AND ("JUnit" OR "pytest" OR "unit test")  
AND ("test generation" OR "automated test generation")
```

```
("large language model" OR "GPT")  
AND ("software testing" OR "unit testing")  
AND ("mutation testing" OR "coverage")
```

Mỗi người search nguồn của mình, rồi ghi lại:

```
database, search_string, date, number_of_results
```

Ví dụ:

```
Google Scholar | String A | 26/05/2026 | 230 results
IEEE Xplore    | String A | 26/05/2026 | 42 results
ACM DL         | String A | 26/05/2026 | 35 results
```

Output:

```
SLR/search_log.xlsx
SLR/raw_records_google_scholar.csv
SLR/raw_records_ieee.csv
SLR/raw_records_acm.csv
SLR/raw_records_semantic_scholar.csv
SLR/raw_records_arxiv_core.csv
```

Giai đoạn 2 — Gộp paper, xóa trùng, lọc paper

Bước 2.1 — Gộp tất cả paper vào một file

Tạo file:

```
SLR/01_all_records.csv
```

Cột nên có:

```
id,title,authors,year,venue,database,search_string,doi,url
```

Ví dụ:

```
1,"An Empirical Study of LLM-based Unit Test Generation","A et al.",2024,"ICSE","ACM","String A",...
2,"ChatGPT for Automated Unit Test Generation","B et al.",2023,"arXiv","arXiv","String B",...
```

Sau đó xóa trùng theo:

```
title
doi
```

```
authors + year
```

Output sau khi xóa trùng:

```
SLR/01_all_records_dedup.csv
```

Bước 2.2 — Viết Inclusion/Exclusion Criteria cho SE1944

Inclusion Criteria, tức tiêu chí giữ paper:

```
IC1: Paper viết tiếng Anh.  
IC2: Xuất bản từ 2018 trở về sau.  
IC3: Liên quan đến LLM/GPT/AI cho test generation hoặc unit test generation.  
IC4: Có thực nghiệm hoặc benchmark rõ ràng.  
IC5: Có metric liên quan đến coverage, branch coverage, mutation score, pass rate hoặc test quality.  
IC6: Có đối tượng test là code Java/Python/function/method/class/project.
```

Exclusion Criteria, tức tiêu chí loại paper:

```
EC1: Duplicate paper.  
EC2: Không tìm được full-text/PDF.  
EC3: Chỉ nói về BDD/Gherkin/API/UI testing, không liên quan unit test.  
EC4: Chỉ nói về test management hoặc test execution, không nói test generation.  
EC5: Không có số liệu thực nghiệm.  
EC6: Không liên quan Software Engineering/testing.
```

Output:

```
SLR/ie_criteria.md
```

Bước 2.3 — Screening vòng 1: Title + Abstract

Mỗi người đọc title + abstract các paper của mình, rồi ghi:

```
INCLUDE  
EXCLUDE  
UNSURE
```

File hướng dẫn có nguyên tắc rất quan trọng: nếu **UNSURE** thì giữ lại cho vòng 2, vì tốt hơn là giữ nhầm một paper hơn là loại nhầm paper tốt.

Cột cần thêm:

```
v1_decision,v1_reason
```

Ví dụ:

```
"ChatGPT for Unit Test Generation",INCLUDE,"liên quan LLM sinh unit test"  
"A Survey of UI Testing",EXCLUDE,"EC3 – UI testing, không phải unit test generation"  
"AI in Software Engineering",UNSURE,"chưa rõ có test generation không"
```

Output:

```
SLR/02_after_screening_v1.csv
```

Bước 2.4 — Screening vòng 2: Full-text

Bây giờ đọc PDF đầy đủ của các paper **INCLUDE** và **UNSURE**.

Đọc nhanh các phần:

```
Abstract  
Introduction  
Method  
Experiment  
Results  
Discussion  
Conclusion
```

Thêm cột:

```
v2_decision,v2_reason
```

Output:

```
SLR/03_final_included.csv
```

Mục tiêu thực tế: còn khoảng **8–15 paper tốt** để làm evidence table.

Giai đoạn 3 — Lập Evidence Table và tìm GAP

Evidence table là bảng quan trọng nhất của phần research. Nó cho thấy nhóm bạn không làm bừa, mà có đọc paper để tìm khoảng trống nghiên cứu.

Tạo file:

SLR/evidence_table.xlsx

Cột nên có:

Cột	Ý nghĩa
Paper	Tên paper
Year	Năm
Model	GPT-4, GPT-3.5, Codex, CodeT5, EvoSuite...
Language	Java/Python/C#/JS
Dataset	HumanEval, Defects4J, project GitHub, custom functions
Baseline	Manual test, EvoSuite, Randoop, developer tests
Metrics	Branch coverage, line coverage, mutation score, pass rate
Main result	Kết quả chính
Limitation	Hạn chế
Useful for our study	Liên quan gì đến SE1944

Ví dụ:

Paper	Model	Language	Baseline	Metric	Limitation
Paper A	GPT-3.5	Java	EvoSuite	Line coverage	Không đo mutation score
Paper B	Codex	Python	Manual tests	Pass rate	Không đo branch coverage
Paper C	GPT-4	Java	Developer tests	Mutation score	Không xét cyclomatic complexity

Sau đó nhóm viết GAP:

Các nghiên cứu trước đã dùng LLM để sinh unit test, nhưng chưa đánh giá rõ GPT-4 trên Java/Python functions có cyclomatic complexity trung bình, đồng thời đo cả branch coverage và mutation score, và so sánh trực tiếp với unit test viết thủ công bởi sinh viên.

Output:

SLR/gap_evidence.md

Giai đoạn 4 — Thiết kế thực nghiệm

Đây là phần “làm thật”.

Bước 4.1 — Chọn ngôn ngữ và tool

Mình khuyên chọn **Java** để dễ làm báo cáo:

```
Language: Java
Unit test framework: JUnit 5
Coverage tool: JaCoCo
Mutation testing tool: PIT / PiTest
Build tool: Maven
```

Vì dễ tài đo branch coverage và mutation score, Java + JaCoCo + PIT rất rõ ràng.

Bước 4.2 — Chuẩn bị code dataset

Nhóm cần một tập hàm Java để test.

Có 3 cách:

Cách	Đánh giá
Lấy từ open-source/simple Java project	Tốt nhất
Tự viết các class nhỏ có logic rõ	Dễ làm nhất cho môn học
Lấy từ bài tập thuật toán/utility functions	Tạm ổn

Với nhóm mới, nên làm:

```
20 hàm Java
Mỗi hàm có cyclomatic complexity khoảng 3–7
Có if/else, switch, loop, boundary cases
Không cần database, không cần gọi API
```

Ví dụ function phù hợp:

```
calculateDiscount()
classifyTriangle()
calculateParkingFee()
validatePassword()
calculateShippingFee()
gradeScore()
calculateTax()
isEligibleForLoan()
```

Mỗi function cần có metadata:

```
function_id
class_name
method_name
description
input
expected_behavior
cyclomatic_complexity
```

Output:

```
dataset/functions/
dataset/metadata.csv
```

Ví dụ metadata:

```
F001,DiscountService,calculateDiscount,"Tính giảm giá",Java,CC=4
F002,TriangleClassifier,classifyTriangle,"Phân loại tam giác",Java,CC=6
```

Giai đoạn 5 — Sinh test bằng AI

Người phụ trách AI sẽ tạo prompt cố định.

Ví dụ prompt:

```
You are a senior Java unit testing engineer.
Generate JUnit 5 unit tests for the following Java method.
Requirements:
- Cover normal cases, boundary cases, and invalid cases.
- Use clear test method names.
- Include assertions.
- Do not modify production code.
- Return only the test class.

[PASTE CLASS CODE HERE]
```

Quy tắc quan trọng:

```
Cùng một prompt cho tất cả functions.
Cùng một model GPT cho tất cả functions.
Không sửa test AI quá nhiều, nếu sửa phải ghi log.
Temperature = 0 nếu dùng API.
```


Output:

```
tests_ai/F001_DiscountServiceGPTTest.java
tests_ai/F002_TriangleClassifierGPTTest.java
...
prompts/prompt_v1.md
prompts/ai_generation_log.xlsx
```

Log cần ghi:

```
function_id
model
prompt_version
date
raw_output_file
compile_status
notes
```

Giai đoạn 6 — Sinh viên viết test thủ công

Đây là baseline.

Để công bằng, sinh viên viết test thủ công **không được xem test của AI**.

Chia như này:

```
Người 1 viết manual test cho F001–F004
Người 2 viết manual test cho F005–F008
Người 3 viết manual test cho F009–F012
Người 4 viết manual test cho F013–F016
Người 5 viết manual test cho F017–F020
```

Quy tắc:

```
Chỉ xem production code và mô tả expected behavior.
Không dùng ChatGPT để viết manual test.
Không xem kết quả coverage trước khi viết bản đầu.
Ghi thời gian viết test nếu muốn đo effort.
```

Output:

```
tests_student/F001_DiscountServiceStudentTest.java
tests_student/F002_TriangleClassifierStudentTest.java
```

```
...  
experiment/manual_test_log.xlsx
```

Giai đoạn 7 — Chạy test và đo kết quả

Với mỗi function, nhóm chạy 2 bộ test:

```
Bộ 1: AI-generated tests  
Bộ 2: Student-written tests
```

Đo 2 metric chính:

```
Branch coverage  
Mutation score
```

Nếu dùng Java

Chạy test:

```
mvn test
```

Chạy coverage:

```
mvn jacoco:report
```

Chạy mutation testing:

```
mvn org.pitest:pitest-maven:mutationCoverage
```

Đọc kết quả:

```
target/site/jacoco/index.html  
target/pit-reports/index.html
```

Trong tài liệu hướng dẫn PiTest của bạn, mutation score được hiểu là **Killed / Total**, mutant **KILLED** nghĩa là test phát hiện được lỗi giả, còn **SURVIVED** nghĩa là test bỏ sót lỗi giả.

Output:

```
results/coverage_ai.csv
results/coverage_student.csv
results/mutation_ai.csv
results/mutation_student.csv
```

Bảng kết quả cuối:

Function	CC	AI Branch	Student Branch	AI Mutation	Student Mutation
F001	4	85%	75%	62%	58%
F002	6	90%	80%	70%	65%
F003	5	78%	82%	55%	60%

Giai đoạn 8 — Phân tích kết quả

Bạn cần trả lời 4 câu:

Câu 1: GPT-4 có đạt branch coverage $\geq 80\%$ không?

Ví dụ:

```
Trong 20 functions, AI đạt branch coverage trung bình 83%.
Có 15/20 functions đạt  $\geq 80\%$ .
```

Câu 2: GPT-4 có đạt mutation score $\geq 60\%$ không?

Ví dụ:

```
AI đạt mutation score trung bình 61%.
Có 12/20 functions đạt  $\geq 60\%$ .
```

Câu 3: AI so với sinh viên thế nào?

Có 3 khả năng:

```
AI tốt hơn sinh viên
AI ngang sinh viên
AI kém sinh viên
```

Ví dụ:

AI có branch coverage cao hơn sinh viên ở 13/20 functions, nhưng mutation score chỉ cao hơn ở 9/20 functions.

Kết luận kiểu này rất hay:

GPT-4 có xu hướng tạo nhiều test case giúp tăng coverage, nhưng chưa chắc assert đủ mạnh để kill mutants tốt hơn sinh viên.

Câu 4: Vì sao có function AI làm kém?

Phân tích theo lỗi:

AI thiếu boundary cases.
AI không hiểu expected behavior.
AI assert quá yếu.
AI chỉ test happy path.
AI test compile lỗi.
AI không mock dependency được.

Output:

results/final_analysis.xlsx
results/charts/
paper/results.md

Giai đoạn 9 — Viết báo cáo/paper

Cấu trúc nên viết theo IMRaD:

1. Introduction
2. Related Work
3. Methodology
4. Experiment Design
5. Results
6. Discussion
7. Threats to Validity
8. Conclusion

Introduction

Nói vấn đề:

Unit testing tốn thời gian.
LLM có thể sinh unit test tự động.
Nhưng cần đánh giá bằng metric khách quan như branch coverage và mutation score.

Related Work

Dựa trên Evidence Table.

Methodology

Mô tả:

PRISMA search
IE criteria
Evidence table
GAP

Experiment Design

Mô tả:

20 Java functions
CC trung bình 3–7
GPT-4 sinh test
Sinh viên viết test thủ công
JaCoCo đo branch coverage
PIT đo mutation score

Results

Bảng số liệu.

Discussion

Giải thích vì sao AI tốt/kém.

Threats to Validity

Phần này rất quan trọng. Ghi các hạn chế:

Dataset nhỏ.
Chỉ dùng Java hoặc Python.
Sinh viên có trình độ khác nhau.
Prompt ảnh hưởng kết quả.

Model GPT có thể thay đổi theo thời gian.
Manual tests có thể không đại diện cho expert tests.

Conclusion

Trả lời RQ:

GPT-4 đạt/không đạt branch coverage $\geq 80\%$.
GPT-4 đạt/không đạt mutation score $\geq 60\%$.
So với sinh viên, GPT-4 tốt hơn/ngang/kém ở điểm nào.

Chia việc cụ thể cho nhóm 5 người

Người 1 — Leader + Methodology

Phụ trách:

Chốt RQ/PICO.
Quản lý folder Google Drive/GitHub.
Tạo template CSV.
Gộp paper.
Xóa duplicate.
Viết Methodology + PRISMA.
Tổng hợp báo cáo cuối.

Deliverables:

docs/pico.md
SLR/01_all_records_dedup.csv
paper/methodology.md
paper/final_report.docx

Người 2 — Literature Search A

Phụ trách:

Search Google Scholar.
Search Semantic Scholar.
Lọc title/abstract.
Tìm PDF.
Tóm tắt 3–5 paper vào Evidence Table.

Deliverables:

```
SLR/raw_records_google_scholar.csv
SLR/raw_records_semantic_scholar.csv
SLR/evidence_table_part_A.xlsx
```

Người 3 — Literature Search B

Phụ trách:

```
Search IEEE Xplore.
Search ACM Digital Library.
Lọc title/abstract.
Tìm PDF bằng Semantic Scholar/CORE.
Tóm tắt 3–5 paper vào Evidence Table.
```

Deliverables:

```
SLR/raw_records_ieee.csv
SLR/raw_records_acm.csv
SLR/evidence_table_part_B.xlsx
```

Người 4 — Dataset + AI Test

Phụ trách:

```
Chuẩn bị 20 Java/Python functions.
Tính cyclomatic complexity.
Viết metadata.
Tạo prompt.
Cho GPT-4 sinh test.
Lưu raw output.
Fix lỗi compile tối thiểu nếu cần và ghi log.
```

Deliverables:

```
dataset/functions/
dataset/metadata.csv
prompts/prompt_v1.md
tests_ai/
experiment/ai_generation_log.xlsx
```

Người 5 — Manual Test + Tool Execution

Phụ trách:

Điều phối sinh viên viết test thủ công.
Tách test AI và test Student.
Chạy JaCoCo/coverage.
Chạy PIT/mutation.
Xuất kết quả CSV.
Vẽ bảng/chart.

Deliverables:

tests_student/
results/coverage_ai.csv
results/coverage_student.csv
results/mutation_ai.csv
results/mutation_student.csv
results/final_results.xlsx

Timeline gợi ý trong 7 ngày

Ngày 1

Chốt PICO.
Chốt Java hay Python.
Tạo Google Drive/GitHub folder.
Viết search strings.
Mỗi người bắt đầu search paper.

Ngày 2

Mỗi người export paper từ nguồn của mình.
Gộp file.
Xóa duplicate.
Lọc vòng 1 title + abstract.

Ngày 3

Tải PDF.
Lọc vòng 2 full-text.
Lập evidence table.
Viết GAP.

Ngày 4

Chuẩn bị 20 functions.
Tính cyclomatic complexity.
Chạy thử Maven/JUnit/JaCoCo/PIT.

Ngày 5

GPT-4 sinh test.
Sinh viên viết manual test.
Lưu toàn bộ test.

Ngày 6

Chạy coverage.
Chạy mutation testing.
Tổng hợp kết quả.
Vẽ bảng so sánh.

Ngày 7

Viết báo cáo.
Viết slide.
Chuẩn bị demo.

Folder nên tạo ngay

```
SE1944_LLM_Unit_Test_Generation/  
├── docs/  
│   ├── pico.md  
│   ├── scope.md  
│   └── research_question.md  
├── SLR/  
│   ├── search_log.xlsx  
│   ├── 01_all_records.csv  
│   ├── 01_all_records_dedup.csv  
│   ├── 02_after_screening_v1.csv  
│   ├── 03_final_included.csv  
│   ├── ie_criteria.md  
│   ├── evidence_table.xlsx  
│   └── gap_evidence.md  
└── dataset/
```

```
| | functions/
| | └─ metadata.csv
└─ prompts/
    └─ prompt_v1.md
└─ tests_ai/
└─ tests_student/
└─ results/
    ├── coverage_ai.csv
    ├── coverage_student.csv
    ├── mutation_ai.csv
    ├── mutation_student.csv
    └─ final_results.xlsx
└─ paper/
    ├── introduction.md
    ├── related_work.md
    ├── methodology.md
    ├── results.md
    ├── discussion.md
    └─ final_report.docx
```

Nói cực ngắn: nhóm 5 người làm gì?

Người 1: Quản lý RQ, PRISMA, báo cáo.

Người 2: Tìm paper Google Scholar/Semantic Scholar.

Người 3: Tìm paper IEEE/ACM.

Người 4: Chuẩn bị code dataset + cho GPT-4 sinh test.

Người 5: Quản lý manual test + chạy JaCoCo/PIT + tổng hợp số liệu.

Kết quả cuối cùng nhóm cần có không chỉ là "tìm được paper", mà là:

1. Evidence Table từ paper.
2. GAP của nhóm.
3. Dataset functions.
4. Test do GPT-4 sinh.
5. Test do sinh viên viết.
6. Bảng branch coverage.
7. Bảng mutation score.
8. So sánh AI vs student.
9. Báo cáo/paper.

Vậy nên hiểu đúng nhất là: **paper search là phần nền; thực nghiệm AI vs sinh viên mới là phần chính để trả lời RQ của SE1944.**